

タイトル候補

バグチケットをAIで全自動解決する — 人の仕事を生成AIに引き継ぐために設計したこと

バグチケットをAIで全自動解決する — 生成AIに人の仕事を任せきるために必要だったこと

バグチケットをAIで全自動解決する — 人がやっていた全工程を生成AIに渡すために設計したこと

バグチケットをAIで全自動解決する — 生成AIに仕事を引き継ぐための設計と、ぶつかった壁

manattan

AIの活用が進んで、エンジニアの働き方が変わってきた

Copilot、Claude Code、Slack AI、Rovo、Gemini、NotebookLM...

気づけば、開発者でなくてもAIを使える時代になった。

自分もClaude CodeやCopilotで日常的にコードを書いている、実装スピードは確実に上がった。

その結果、エンジニアの働き方も変わってきている：

- 案件を並行で動かせるようになった
- 実装だけでなく、PdMとの仕様整理や上流の設計にも時間を使うようになった

ただ、できることが増えた分だけ、別の負荷も増えている。

速くなった結果、起きたこと

実装が速くなった分、並行で抱える案件が増えた。

- 修正中に別のバグ報告が上がる、Slackで問い合わせが飛んでくる
- Jiraを見て、ログを見て、GitHubを見て、エディタに戻って、またSlack...
- Slack AIの要約、Jiraの調査メモ、Copilotの出力、それぞれバラバラに出てくるが、つなげるのは人間

AIが速く書けるなら、もっと回せるでしょ？ — 自分でもそう思うし、周りからもそう見られる。

ボトルネックは「コードを書くこと」の外にある

コンテキストスイッチ、割り込み対応、散在する情報の整理。

「コードを書く前後」の作業が、抜け漏れや非効率の原因になっている。

AIにもっと任せたいなら、「コードを書く」以外の部分も含めて、**AIが動ける環境・実行基盤を整える必要がある。**

その取り組みのひとつとして、バグ報告の初動対応を自動化するBanchoを作っている。

目指す状態

	いま（人間が全部やる）	これから（AIが初動を担う）
バグの検知	人間が気づく（遅延あり）	自動検知・即時振り分け
原因の特定	どこで起きたか人が調べる	AIが候補を絞って提示
情報収集	チケット・ログ・コードを人が巡回	AIが各ツールから自動で集める
報告の起票	手動で転記・整理	調査結果つきで自動作成
初動の所要時間	30分～	数分（人間はレビューだけ）

Banchoの概要

バグ管理ツール（Jira）にチケットが立つと、AIが自動で初動調査し、開発タスク（GitHub Issue）を作成する仕組み。

1. バグ報告が来る → AIが自動で検知
2. どのサービスで起きたか、どの環境かを推定
3. チケット情報・ログ・ソースコードをAIが自動で集める
4. 原因の仮説と修正方針を作成
5. GitHub Issueを作成 → Copilotにメンション → 修正PRへ

AIと各ツールの連携にはMCP（Model Context Protocol）を採用。

Jira・ログ基盤・GitHubへのアクセスを、AIが使える「道具」として統一的に定義できる。

アーキテクチャ

Jiraでバグ報告が作成される



Webhookで通知を受信（署名検証で不正リクエストを排除）



対象チケットか判定 → AIが分類 → 対象外は除外



AIエージェントが調査開始



Jira から チケットの詳細を取得



ログ基盤（Grafana/Loki）から関連ログを取得



GitHub から ソースコード情報を取得



GitHub Issueを作成 / Jiraにコメントで結果を返す

AIがアクセスしているサービス・環境の範囲は、設定ファイル（YAML）で一元管理している。

AIには「自由に考えさせない」が鉄則だった

AIに「どのサービスを調べるか」「どの環境を見るか」を自由に判断させると、存在しないものを参照したり、触ってはいけない環境にアクセスしてしまう。

やったのは、**AIが選べる範囲をあらかじめ定義して、その中から選ばせること。**

```
# サービスカタログ（設定ファイル）の例
repositories:
  - service: bancho
    repository: manattan/bancho # 調査対象のコード
environments:
  qa:
    namespace: bancho-qa # 見ていい環境
    grafana_datasource: loki-qa # 見ていいログ
```

この設定ファイルが「AIに何を見せていいか」の唯一の定義。AIが使えるツールの範囲もここで制限している。

技術的には、ほぼ動く

通知の受信、チケットの振り分け、AIによる調査、結果の出力。一通りの流れは実装できた。

ただ、「動く」と「業務◆◆◆使える」の間には、思っていたより大きな溝があった。

ここから先は、コードでは解決できない問題の話。

壁①：ツールごとに環境の境界線が違う

AIエージェントは複数のツールを横断して動く。ところが、ツールごとに環境分離の粒度が違った。

- **Jira**: 環境分離されていない。dev環境から既にprod相当のチケットが見える
- **ログ (Loki)** : 環境分離されている。devからprodのログは見れない
- **Webhook**: prod相当のイベントがdevに飛ぶこと自体は許容。ただしアプリ側で制御が必要

人間が手動で調査するときは、頭の中で「これはdevの話」「これはprodの話」と切り替えている。AIエージェントにはその暗黙の境界を設計として定義しないといけない。

しかもAtlassian側にWebhookを環境ごとに振り分ける仕組みがなく、アプリ側でのルーティング設計が必要になった。

壁②：AIに権限を渡すと、設計判断が一気に表面化する

人間が手動でやっていたときは暗黙的に済んでいたことが、AIエージェントに任せた途端に明示的な設計判断になる。

- **誰の権限で動く？** → ユーザに紐づかないサーバプロセスとしてAIが動く場合、既存の認証方式がそのまま使えない
- **どこまで見せる？** → 組織の全repoに権限は渡せない。対象repoを明示し、権限を絞る運用が前提
- **AIの出力を信用できる？** → 調査結果に生ログが混入するリスク。dev/qaなら許容できるが、prodなら別設計が必要
- **どこに書く？** → Slack連携は権限昇格の問題があり、privateチャンネル前提になる

人間なら無意識にやっている「ここは書かない」「ここは見ない」という判断を、AIには設計として教える必要がある。

壁③：AIエージェントの導入は、技術だけでは完結しない

新しいMCPやAPIを使いたいとき、開発者としてはコードを書けば動く。でもそれを業務に載せるには、関係チームとの合意が先に必要だった。

- **AIがチケットを読んでいい範囲は？** → 秘匿情報を含むチケットもある。誰の権限でどこまで読むのか
- **新しいツール連携の導入判断は誰がする？** → Rovo MCPを使いたいけど、IT戦略チームの許可が必要
- **監査やトレーサビリティは？** → AIが何を読み、何を書いたかを追跡できる仕組みが求められる

エンジニアが「技術的にできる」と思っていることと、組織として「やっていい」の間には、想像以上に距離があった。

実装コードを書く時間より、この調整の方が何倍も時間がかかった。

いきなり全自動にしない

AIエージェントは「一気に任せる」より「少しずつ信頼を広げる」方がうまくいく。

1. **まず安全な範囲で動かす** → 開発・QA環境だけで、調査とIssue作成まで（いまここ）
2. **人間のレビューを挟む** → AIが出した修正方針を、人間が確認してからPRへ
3. **実績ができれば広げる** → 本番環境、他チーム、他の業務領域へ

また、チケットからサービスや環境を推定する部分はAIの精度に依存するので、実際に動かしてみないとわからないことも多い。だからこそ、小さく試して「これなら任せられる」という信頼を積む方が、結果的に速い。

AIの実行基盤を整えるということ

Banchoは「バグ初動対応」という一つの領域の話。でも、見えてきた課題はもっと広い。

- 権限・環境・監査の設計は、どのAI活用でも必ず出てくる
- 「技術的にできる」と「組織としてやっていい」のギャップは、活用範囲が広がるほど大きくなる
- このギャップを埋める仕組みを作ることが、AIの実行基盤を整えるということ

これはSlackボットや社内ツールなど、生成AIに人の仕事を任せる仕組みを作るなら、どこでもぶつかる壁でもある。

まとめ

AIがコードを書く時代になって、エンジニアの仕事は「書くこと」から「**AIが動ける状態を作ること**」に移りつつある。

そのために必要なのは、AIの性能よりも、**権限・環境・組織の設計を含めた実行基盤**。

これからAIを業務に組み込もうとしている方の参考になれば幸いです。